CSE 574A: Intro to Machine Learning
Homework 7

Digit Classification Using Backpropagation

Submitted By: Viral Koshti
Student ID: 50761354

1 Objective and Dataset

The objective is to implement a feedforward neural network from scratch to classify handwritten digits 0–9 using backpropagation. The MNIST dataset contains 60,000 training images and 10,000 test images. Each grayscale image has $28 \times 28 = 784$ pixels. Images are flattened into 784-dimensional vectors, and each pixel intensity $p \in \{0, \ldots, 255\}$ is normalized as $x = p/255$. The training images are used to update model parameters. All 10,000 test images are used to report performance, and none participate in gradient updates.

2 Network Architecture and Output Encoding

The selected model is a fully connected multilayer perceptron (MLP):

$$\boxed{\text{784 inputs} \longrightarrow \text{128 ReLU hidden units} \longrightarrow \text{10 softmax outputs}}.$$

The ten outputs represent the digits $0, 1, \ldots, 9$ using one-hot target vectors. For example, the target for digit 3 is

$$\boldsymbol{y} = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]^{\mathsf{T}}.$$

The predicted digit is the index of the largest output probability, $\hat{y} = \arg\max_{k \in \{0,\ldots,9\}} P_k$. The model uses two weight matrices and two bias vectors:

$$W_1 \in \mathbb{R}^{784 \times 128}, \quad b_1 \in \mathbb{R}^{128}, \qquad W_2 \in \mathbb{R}^{128 \times 10}, \quad b_2 \in \mathbb{R}^{10}.$$

The total number of trainable parameters is

$$784(128) + 128 + 128(10) + 10 = \boxed{101{,}770}.$$

Weights are initialized using He initialization and biases start at zero. The same learning rate is applied to every neuron in both trainable layers at a given epoch.

3 Forward Propagation, Error Functions, and Backpropagation

For a mini-batch $X \in \mathbb{R}^{B \times 784}$, the hidden and output layers are calculated as

$$Z_1 = XW_1 + b_1, \qquad H = \text{ReLU}(Z_1) = \max(0, Z_1), \tag{1}$$

$$Z_2 = HW_2 + b_2, \qquad P_{ik} = \frac{\exp(Z_{2,ik} - m_i)}{\sum_{j=1}^{10} \exp(Z_{2,ij} - m_i)}, \qquad m_i = \max_j Z_{2,ij}. \tag{2}$$

Subtracting m_i makes the softmax computation numerically stable. For the one-hot target matrix Y, training minimizes mean multiclass cross-entropy with L2 weight regularization:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \sum_{k=1}^{10} Y_{ik} \log(P_{ik}) + \frac{\lambda}{2} \left(\|W_1\|_F^2 + \|W_2\|_F^2 \right).$$

The output-layer error signal, gradients, and hidden-layer error signal follow from the chain rule:

$$G_2 = \frac{P - Y}{B}, \qquad \nabla W_2 = H^{\mathsf{T}} G_2 + \lambda W_2, \qquad \nabla b_2 = \sum_{i=1}^{B} G_{2,i:}, \tag{3}$$

$$G_1 = (G_2 W_2^{\mathsf{T}}) \odot \mathbb{1}[Z_1 > 0], \qquad \nabla W_1 = X^{\mathsf{T}} G_1 + \lambda W_1, \qquad \nabla b_1 = \sum_{i=1}^{B} G_{1,i:}. \tag{4}$$

Here $\odot$ denotes elementwise multiplication and $\mathbb{1}[Z_1 > 0]$ is the derivative of ReLU, with zero used at nonpositive inputs. Gradient descent updates each parameter by

$$W_\ell \leftarrow W_\ell - \eta \nabla W_\ell, \qquad b_\ell \leftarrow b_\ell - \eta \nabla b_\ell, \qquad \ell \in \{1, 2\}.$$

The assignment also requires mean squared error (MSE) plots, so MSE is measured independently of the training loss:

$$\text{MSE} = \frac{1}{10N} \sum_{i=1}^{N} \sum_{k=1}^{10} (Y_{ik} - P_{ik})^2, \qquad \text{classification errors} = \sum_{i=1}^{N} \mathbb{1}[\hat{y}_i \neq y_i].$$

4 Hyperparameters and Design Reasoning

Setting	Value	Reason
Hidden layers / units	1 / 128	Nonlinear features with manageable cost; compared with a smaller 64-unit model.
Hidden / output activation	ReLU / softmax	ReLU supports nonlinear features; softmax produces ten class probabilities.
Mini-batch size	128	More stable updates than single-example training, while remaining computationally practical.
Initial learning rate η_0	0.10	A moderate starting step; compared with 0.05 in a controlled run.
Learning-rate schedule	$\eta_e = 0.10(0.95)^{e-1}$	Decreases the step size gradually across epochs.
L2 coefficient λ	10^{-4}	Penalizes excessively large weights.
Epochs / random seed	15 / 42	Fifteen passes through the training set with repeatable initialization.

One hidden layer was chosen because it offers nonlinear modeling capacity while keeping the forward and backward passes straightforward to implement and inspect. No two-hidden-layer model was measured, so this experiment does not establish that a deeper model would perform worse. Similarly, the 128-unit width is supported by the comparison below, not a claim of global optimality. No dropout or momentum was used.

5 Design Process and Controlled Experiments

After checking that the forward pass, gradient shapes, and loss calculation worked on a smaller sample, three configurations were trained on all 60,000 training examples for 10 epochs each. The batch size, learning-rate decay, regularization, and random seed were held constant. The differences below are measured *training* outcomes at epoch 10; separate test accuracies were not recorded for the two alternate configurations.

Configuration	Hidden	η_0	Accuracy	Errors	MSE
A: smaller width	64	0.10	96.69%	1,983	0.005246
B: selected baseline	128	0.10	**97.23%**	**1,659**	**0.004447**
C: slower learning	128	0.05	95.61%	2,636	0.006892

With the same initial learning rate, increasing the hidden width from 64 to 128 improved training accuracy by 0.54 percentage points and reduced training mistakes by 324. Lowering the initial learning rate to 0.05 led to 977 more mistakes after the same 10 epochs. This suggests slower convergence over this training budget, not that the lower learning rate is necessarily inferior if given more epochs. All three runs trained successfully; no numerical divergence was observed. The selected 128-unit configuration was carried forward for full test-set evaluation.

6 Learning Curves and Results

The selected 784–128–10 model was trained for 15 epochs. The full training set of 60,000 examples was used for gradient updates, while the separate 10,000-image test set was evaluated without gradient updates. Figures 1 and 2 display the observed results over all 15 epochs.

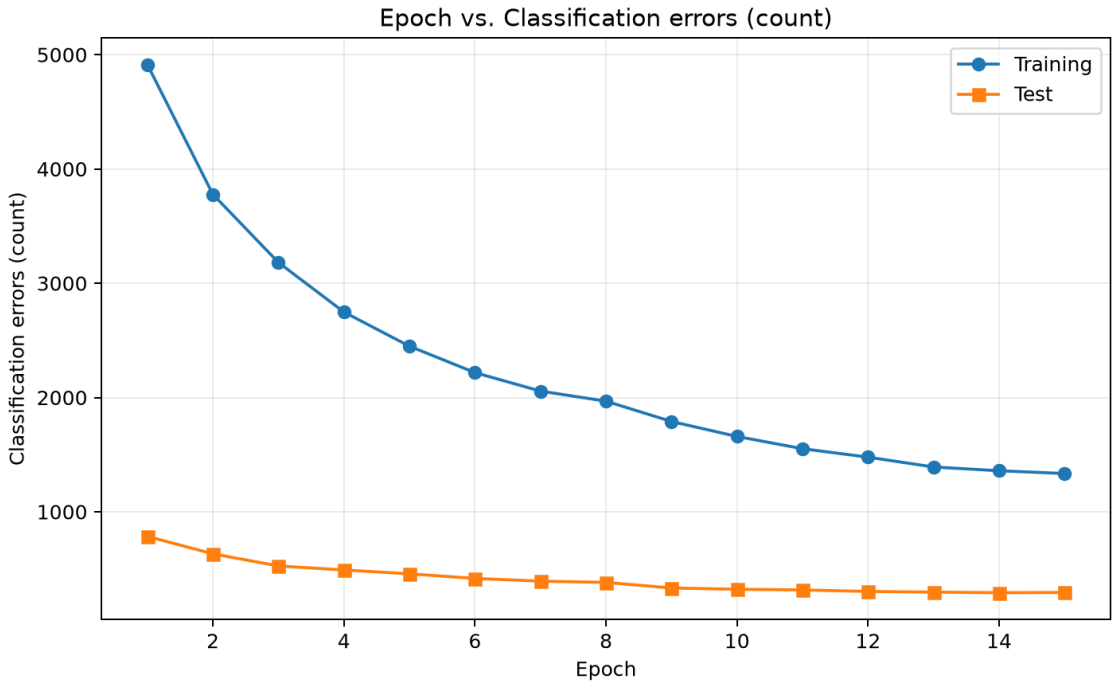


Figure 1: Classification errors versus epoch, using the full 60,000-image training set and 10,000-image test set. Because the datasets have different sizes, their raw error counts are not directly comparable as accuracy measures.

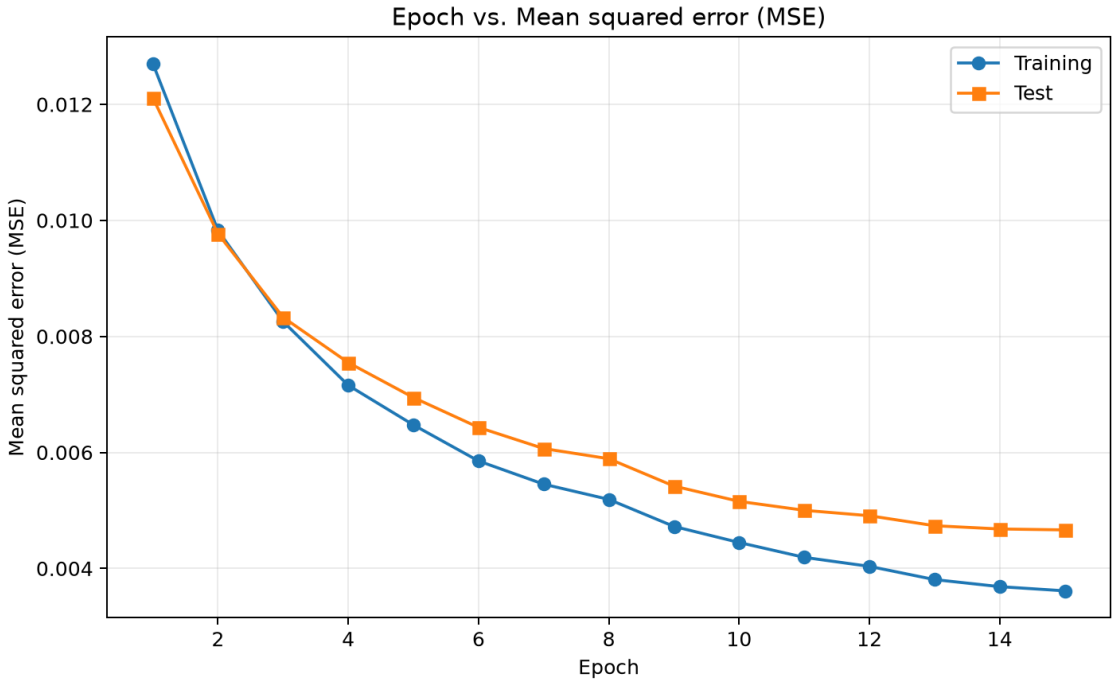


Figure 2: Mean squared error versus epoch for training and test images. Both curves use the same MSE definition.

Table 1: Selected checkpoints from the actual 15-epoch training run.

Epoch	Train acc.	Train errors	Train MSE	Test acc.	Test errors	Test MSE
1	91.81%	4,914	0.012706	92.15%	785	0.012114
4	95.42%	2,748	0.007156	95.09%	491	0.007548
10	97.23%	1,659	0.004447	96.78%	322	0.005156
14	97.73%	1,359	0.003686	**97.08%**	**292**	0.004680
15	**97.78%**	**1,335**	**0.003613**	97.06%	294	**0.004664**

Table 2: Results at the final epoch (epoch 15).

Metric	Training (60,000)	Test (10,000)
Correct predictions	58,665	9,706
Incorrect predictions	1,335	294
Accuracy	97.78%	97.06%
Mean squared error	0.003613	0.004664

7 Discussion and Reproducibility

During the first epoch, the model misclassified 4,914 training images and 785 test images. By epoch 15, these counts had fallen to 1,335 and 294, respectively. The test error count dropped below 500 by epoch 4, corresponding to 95.09% test accuracy. Thus, the model exceeded the homework's approximate 95% target early in training and continued to improve afterward.

At epoch 14, the model achieved its *highest observed test accuracy* of 97.08% (292 errors). Epoch 15 achieved 97.06% (294 errors), a small decrease of 0.02 percentage points, even as the test MSE decreased from 0.004680 to 0.004664. Therefore, epoch 15 is the reported final trained model, while epoch 14 is separately identified as the best observed test-accuracy checkpoint. No general conclusion about the optimal training duration can be made from this one run.

The final training accuracy is 97.78%, versus 97.06% on the test set: a 0.72-percentage-point gap. The higher test MSE (0.004664 versus 0.003613) is consistent with some generalization difference, although test performance remains above the target. The observed decline in MSE over 15 epochs supports that the matrix-based backpropagation updates were effective. Neither dropout nor momentum was used.

The network computations were implemented using NumPy array and matrix operations, including the forward pass, backpropagation gradients, and parameter updates. Python loops control epochs and mini-batches, not individual neuron or weight calculations. Matplotlib was used to generate the required plots. No external neural-network or machine-learning framework was used. The full 15-epoch measurements are also provided in `metrics.csv`. The test set was not used to update the weights; reporting the highest observed test checkpoint should not be confused with selecting or validating hyperparameters on an independent validation set.

References

1. CSE 574A, *Homework 7: Digit Classification Based on Neural Networks*, Fall 2026.
2. CSE 574A, Lecture 8, *Introduction to Neural Networks.*
3. CSE 574A, Lecture 11, *Supervised Learning.*
4. CSE 574A, Lecture 12, *Backpropagation Algorithm & Multi-layer Neural Networks.*